

아래 기준을 **NSIGHT** 애플리케이션 네이밍 원칙 **v1.0**으로 잡으면 됩니다.

핵심은 업무코드 → **Context** → **WAR** → **Package** → **ServiceId** → **Handler** → **Facade** → **Service** → **Rule** → **DAO/Mapper** → **MyBatis XML** → **SQL ID** → 거래로그/오류코드가 하나의 선으로 이어지게 만드는 것입니다.

현재 GitHub 기준 **nsight-tcf-framework**는 HTTP/JSON 표준 전문을 TCF로 처리하는 멀티 모듈 구조이고, 공통 파이프라인은 Header 검증, 세션, 권한, 거래통제, Timeout, 로깅, 응답 조립을 STF/TCF/ETF가 담당하도록 정리되어 있습니다. ([GitHub](#)) 또한 업무 모듈은 **entry/handler** → **entry/facade** → **application/service** → **application/rule** → **persistence/dao|mapper** 흐름을 표준 패키지 규약으로 두고 있습니다. ([GitHub](#)) MyBatis 표준서에서도 MyBatis는 DAO/Mapper 계층에 위치하고, SQL은 Mapper XML에 두며, 업무별 Mapper 분리와 RDW/ADW Mapper 구분, SQL ID 표준화를 원칙으로 두고 있습니다.

1. 최상위 네이밍 원칙

원칙	기준	예시
업무코드 우선	모든 이름은 업무코드를 기준으로 시작한다	SV, OM, MG
하나의 업무코드는 하나의 Context를 가진다	URL, WAR, Package 가 같은 업무코드를 사용한다	/sv, sv.war, .sv
실행 기준은 URL이 아니라 ServiceId	TCF Dispatcher는 serviceId 로 Handler를 찾는다	SV.Customer.selectSummary
Java Class는 PascalCase	첫 글자 대문자, 의미 있는 이름 사용	SvCustomerInquiryHandler

Java Method/Field는 lowerCamelCase	동사 + 업무대상 형태	<code>selectCustomerSummary()</code>
Package는 lowercase	업무코드와 계층을 소문자로 구성	<code>com.nh.nsight.marketing.sv.entry.handler</code>
DB 객체는 UPPER_SNAKE_CASE	테이블, 컬럼, 인덱스, 시퀀스 통일	<code>TCF_TRANSACTION_LOG</code>
MyBatis SQL ID는 추적 가능하게	업무코드, 업무대상, 행위를 연결	<code>SV.Customer.selectSummary</code>
약어 남용 금지	Svc, Mgr, Proc 같은 불명확한 약어 금지	Service, Manager, Processor 사용
운영 추적성 우선	로그, 거래코드, 오류코드에서 원 업무를 바로 알 수 있어야 함	E-SV-BIZ-0001

2. 업무코드 기준 네이밍

현재 GitHub `settings.gradle`에는 `tcf-util`, `tcf-core`, `tcf-web`, `tcf-cache`, `tcf-ui`, `tcf-uj`, `tcf-batch`, `tcf-om`, `tcf-jwt`, `tcf-gateway`와 업무 서비스 `ic-service`, `pc-service`, `ms-service`, `sv-service`, `pd-service`, `eb-service`, `ep-service`, `ss-service`, `mg-service`, `om-service`가 포함되어 있습니다. ([GitHub](#)) 따라서 현재 소스 기준 업무코드는 우선 아래처럼 잡고, 향후 17개 업무로 확장할 때도 같은 규칙을 적용합니다.

업무코드	업무명	Context	WAR	Gradle Module	Java Prefix	Package
------	-----	---------	-----	---------------	-------------	---------

IC	Integration Customer	/ic	ic.war	ic-service	Ic	ic
PC	Private Customer	/pc	pc.war	pc-service	Pc	pc
MS	Mini Single View	/ms	ms.war	ms-service	Ms	ms
SV	Single View	/sv	sv.war	sv-service	Sv	sv
PD	Product	/pd	pd.war	pd-service	Pd	pd
EB	EBM	/eb	eb.war	eb-service	Eb	eb
EP	Event Processing	/ep	ep.war	ep-service	Ep	ep
SS	Sales Support	/ss	ss.war	ss-service	Ss	ss
MG	Message	/mg	mg.war	mg-service	Mg	mg
OM	Operation Management	/om	om.war	tcf-om	Om	om

GW	Gateway	/gw	gw.war	tcf-gateway	Gateway	gateway
JWT	JWT Auth	/jwt	jwt.war	tcf-jwt	Jwt	jwt
UI	UI Relay	/ui	ui.war	tcf-ui	Ui	ui
UJ	Gateway Test UI	/uj	uj.war	tcf-uj	Uj	uj
BATCH	Batch	/batch	batch.war	tcf-batch	Batch	batch

향후 CC, BC, CM, BP, BD, CS, CT 등을 추가할 경우에도 동일하게 `cc-service`, `/cc`, `cc.war`, `CcXxxHandler`, `com.nh.nsight.marketing.cc...` 형식을 사용합니다.

3. 표준 Package 구조

현재 README는 업무 WAR 패키지 규약을 6계층으로 제시하고 있으며, 예시 루트는 `com.nh.nsight.marketing.{업무}` 또는 인증·게이트웨이 성격에 따라 `com.nh.nsight.auth.jwt`, `com.nh.nsight.gateway`를 사용하도록 정리되어 있습니다. ([GitHub](#)) 실제 GitHub 트리에서도 `sv-service`의 Java package는 `com/nh/nsight/marketing/sv`, `tcf-core`는 `com/nh/nsight/tcf/core` 구조를 사용합니다. ([GitHub](#)) ([GitHub](#))

`com.nh.nsight.marketing.{businessCode}`

└─ entry

| └─ handler

| └─ facade

- | └─ web
- └─ application
 - | └─ service
 - | └─ rule
 - | └─ scheduler
- └─ persistence
 - | └─ dao
 - | └─ mapper
- └─ client
- └─ config
- └─ support

Package	역할	클래스 예시
entry.handler	TCF <code>TransactionHandler</code> 구현	<code>SvCustomerSummaryInquiryHandler</code>
entry.facade	Handler와 Service 사이 유스케이스 조립	<code>SvCustomerFacade</code>
entry.web	별도 REST Controller, Filter 필요 시	<code>SvFileController</code>
application.service	업무 처리 절차	<code>SvCustomerService</code>
application.rule	업무 규칙, 판단 로직	<code>SvCustomerGradeRule</code>

application.scheduler	스케줄러	EbEventCollectScheduler
persistence.dao	DB 접근 조립, Mapper 호출	SvCustomerDao
persistence.mapper	MyBatis Mapper Interface	SvCustomerMapper
client	외부 API/WAS 호출	SvCustomerProfileClient
config	Spring 설정, Properties	SvDataSourceConfig
support	상수, 유틸, 도메인 헬퍼	SvConstants

4. Java 클래스 네이밍

계층	규칙	예시
Handler	{업무Prefix}{업무대상}{행위}Handler	SvCustomerSummaryInquiryHandler
Facade	{업무Prefix}{업무대상}Facade	SvCustomerFacade

Service	{업무Prefix}{업무대상}Service	SvCustomerService
Rule	{업무Prefix}{업무대상}{규칙명}Rule	SvCustomerEligibilityRule
DAO	{업무Prefix}{업무대상}Dao	SvCustomerDao
Mapper	{업무Prefix}{업무대상}Mapper	SvCustomerMapper
Request DTO	{업무대상}{행위}Request	CustomerSummaryInquiryRequest
Response DTO	{업무대상}{행위}Response	CustomerSummaryInquiryResponse
Result DTO	{업무대상}{결과명}Result	CustomerSummaryResult
Command DTO	{업무대상}{행위}Command	CampaignCreateCommand
Criteria DTO	{업무대상}SearchCriteria	CustomerSearchCriteria
Config	{업무Prefix}{대상}Config	SvMyBatisConfig
Properties	{업무Prefix}{대상}Properties	SvTimeoutProperties

Validator	{업무Prefix}{대상}Validato r	SvCustomerValidator
-----------	-----------------------------	---------------------

Client	{업무Prefix}{외부대상}Clien t	SvProfileClient
--------	----------------------------	-----------------

Handler 이름은 ServiceId와 연결되어야 합니다.

ServiceId	Handler
SV.Customer.select Summary	SvCustomerSummaryInqui ryHandler
SV.Customer.select Detail	SvCustomerDetailInquir yHandler
CM.Campaign.create	CmCampaignCreateHandle r
OM.User.inquiry	OmUserInquiryHandler
OM.ServiceCatalog. save	OmServiceCatalogSaveHa ndler
JWT.Auth.ssoIssue	JwtAuthSsoIssueHandler

5. ServiceId / 거래코드 네이밍

README에서도 ServiceId 형식은 {업무코드}.{업무명}.{처리유형}으로 제시하고 있으며, 예시로 SV.Customer.selectSummary, OM.User.inquiry를 사용합니다. ([GitHub](#))

5.1 ServiceId

{BUSINESS_CODE}.{BusinessObject}.{Action}

구분	규칙	예시
BUSINESS_CODE	대문자 업무코드	SV
BusinessObject	업무대상 PascalCase	Customer, Campaign, User
Action	처리 행위 lowerCamelCase	selectSummary, create, update

처리유형	표준 Action	설명
단건 조회	selectDetail	상세 조회
목록 조회	selectList	목록 조회
요약 조회	selectSummary	요약 정보 조회

존재 확인	<code>exists</code>	존재 여부 확인
건수 조회	<code>count</code>	건수 조회
등록	<code>create</code>	신규 생성
수정	<code>update</code>	기존 데이터 수정
삭제	<code>delete</code>	논리 삭제 우선
저장	<code>save</code>	등록/수정 통합
발급	<code>issue</code>	토큰, 번호 등 발급
검증	<code>verify</code>	토큰, 권한 등 검증
실행	<code>execute</code>	배치, 거래 실행
재처리	<code>retry</code>	실패 거래 재처리
다운로드	<code>download</code>	파일 다운로드
업로드	<code>upload</code>	파일 업로드

5.2 거래코드

{BUSINESS_CODE}-{TYPE}-{NNNN}

TYPE	의미	예시
INQ	조회	SV-INQ-0001
REG	등록	CM-REG-0001
UPD	수정	OM-UPD-0001
DEL	삭제	OM-DEL-0001
EXE	실행	BT-EXE-0001
DWN	다운로드	UD-DWN-0001
UPL	업로드	UD-UPL-0001
AUTH	인증	JWT-AUTH-0001

6. MyBatis 네이밍 원칙

NSIGHT MyBatis 표준은 SQL을 Java 코드에 직접 작성하지 않고 Mapper XML에 두며, 업무별 Mapper를 분리하고, RDW/ADW Mapper를 구분하며, Query Timeout과 SQL ID 표준화를 원칙으로 둡니다.

6.1 Mapper Interface

{업무Prefix}{업무대상}Mapper

구분	예시
고객 조회	<code>SvCustomerMapper</code>
캠페인	<code>CmCampaignMapper</code>
사용자관리	<code>OmUserMapper</code>
서비스카탈로그	<code>OmServiceCatalogMapper</code>
거래로그	<code>OmTransactionLogMapper</code>
JWT 토큰	<code>JwtTokenMapper</code>

권장 위치:

`src/main/java/com/nh/nsight/marketing/sv/persistence/mapper/SvCustomerMapper.java`

6.2 Mapper XML

mapper/{businessCode}/{MapperName}.xml

구분	표준 경로
SV 고객 Mapper	<code>classpath:/mapper/sv/SvCustomerMapper.xml</code>
OM 사용자 Mapper	<code>classpath:/mapper/om/OmUserMapper.xml</code>
JWT 토큰 Mapper	<code>classpath:/mapper/jwt/JwtTokenMapper.xml</code>

6.3 XML namespace

```
<mapper  
namespace="com.nh.naight.marketing.sv.persistence.mapper.SvCustomerMapper">
```

namespace는 반드시 Java Mapper Interface FQCN과 일치시킵니다.

6.4 SQL ID

MyBatis XML의 SQL ID는 Java Mapper Method와 1:1로 맞춥니다.

SQL 유형	Method / SQL ID	예시
단건 조회	<code>select{대상}Detail</code>	<code>selectCustomerDetail</code>

목록 조회	<code>select{대상}List</code>	<code>selectCustomerList</code>
요약 조회	<code>select{대상}Summary</code>	<code>selectCustomerSummary</code>
건수 조회	<code>count{대상}</code>	<code>countCustomerList</code>
존재 확인	<code>exists{대상}</code>	<code>existsCustomer</code>
등록	<code>insert{대상}</code>	<code>insertCampaign</code>
수정	<code>update{대상}</code>	<code>updateCampaign</code>
삭제	<code>delete{대상}</code>	<code>deleteCampaign</code>
논리삭제	<code>update{대상}DeleteYn</code>	<code>updateUserDeleteYn</code>
병합	<code>merge{대상}</code>	<code>mergeServiceCatalog</code>

예시:

```
public interface SvCustomerMapper {

    CustomerSummaryResult selectCustomerSummary(CustomerSummaryInquiryRequest request);

    CustomerDetailResult selectCustomerDetail(CustomerDetailInquiryRequest request);

    List<CustomerResult> selectCustomerList(CustomerSearchCriteria criteria);
```

```

        int countCustomerList(CustomerSearchCriteria criteria);
    }

<mapper
namespace="com.nh.nsight.marketing.sv.persistence.mapper.SvCustomerMapper">

    <select id="selectCustomerSummary"

        parameterType="CustomerSummaryInquiryRequest"

        resultType="CustomerSummaryResult">

        SELECT

            CUSTOMER_ID,

            CUSTOMER_NAME,

            CUSTOMER_GRADE

        FROM RDW_CUSTOMER

        WHERE CUSTOMER_ID = #{customerId}

    </select>

</mapper>

```

7. RDW / ADW / SESSIONDB / LOGDB Mapper 분리

HikariCP 표준서에서도 NSIGHT는 RDW, ADW, SESSIONDB를 분리해서 사용하고, 각 DB 목적이 다르므로 Pool도 분리해야 한다고 정리되어 있습니다. MyBatis도 이 기준을 따라 Mapper를 분리합니다.

DB	Mapper 명명	예시	용도
RDW	<code>{업무Prefix}{대상}RdwMapper</code>	<code>SvCustomerRdwMapper</code>	실시간 조회
ADW	<code>{업무Prefix}{대상}AdwMapper</code>	<code>CmCampaignAdwMapper</code>	분석/집계
SESSIONDB	<code>{업무Prefix}{대상}SessionMapper</code>	<code>OmSessionMapper</code>	세션 관리
LOGDB	<code>{업무Prefix}{대상}LogMapper</code>	<code>OmTransactionLogMapper</code>	거래로그/감사로그
OMDB	<code>{업무Prefix}{대상}Mapper</code>	<code>OmUserMapper</code>	운영관리 기준정보

같은 업무대상이라도 RDW 조회와 ADW 분석은 Mapper를 섞지 않습니다.

좋은:

`SvCustomerRdwMapper`

`SvCustomerAdwMapper`

나쁨:

`SvCustomerMapper` 안에 RDW SQL + ADW SQL 혼재

8. DB 객체 네이밍

객체	규칙	예시
업무 테이블	{업무코드}_{업무대상}	SV_CUSTOMER_SUMMARY
TCF 공통 테이블	TCF_{대상}	TCF_TRANSACTION_LOG
OM 관리 테이블	OM_{대상}	OM_USER, OM_SERVICE_CATALOG
JWT 테이블	JWT_{대상}	JWT_TOKEN, JWT_REFRESH_TOKEN
UD 파일 테이블	UD_{대상}	UD_FILE_META
컬럼	UPPER_SNAKE_CASE	SERVICE_ID, TRANSACTION_CODE
PK	PK_{TABLE}	PK_OM_USER
FK	FK_{FROM}_{TO}	FK_OM_USER_AUTH_GROUP
UK	UK_{TABLE}_{컬럼}	UK_OM_USER_LOGIN_ID
Index	IX_{TABLE}_{컬럼}	IX_TCF_TX_LOG_GUID

Sequence	SQ_{TABLE}	SQ_TCF_TRANSACTION_LOG
View	VW_{업무}_{대상}	VW_SV_CUSTOMER_SUMMARY

공통 컬럼은 아래처럼 통일합니다.

컬럼	의미
CREATED_AT	생성일시
CREATED_BY	생성자
UPDATED_AT	수정일시
UPDATED_BY	수정자
USE_YN	사용 여부
DELETE_YN	삭제 여부
VERSION_NO	낙관적 Lock 또는 변경 버전

DESCRIPTION 설명
ION

9. URL / API / Controller 네이밍

NSIGHT는 REST 리소스 방식보다 `POST /{businessCode}/online` + JSON 표준 전문 + ServiceId Dispatcher 방식이 중심입니다. README도 `POST /{businessCode}/online`으로 `StandardRequest(header + body)`를 수신하고 `TransactionDispatcher` → `TransactionHandler(serviceId)` 흐름으로 처리한다고 설명합니다. ([GitHub](#))

구분	표준	예시
업무 온라인 URL	<code>/businessCode/online</code>	<code>/sv/online</code>
OM 온라인 URL	<code>/om/online</code>	<code>/om/online</code>
Gateway URL	<code>/gw/businessCode/online</code> 또는 <code>/api/tcf/businessCode/online</code>	<code>/gw/sv/online</code>
파일 API	<code>/ud/files</code>	<code>/om/ud/files</code> 또는 <code>/ud/files</code>
Actuator	<code>/context/actuator/health</code>	<code>/sv/actuator/health</code>

Controller는 업무별로 많이 만들지 않고, TCF 공통 진입점 중심으로 둡니다.

구분	클래스명
공통 온라인 Controller	OnlineTransactionController
Gateway Controller	GatewayOnlineController
파일 Controller	UdFileController
OM Admin Relay Controller	OmAdminRelayController

10. 오류코드 네이밍

E-{DOMAIN}-{CATEGORY}-{NNNN}

구분	예시	의미
Header 오류	E-TCF-HDR-001	필수 Header 누락
거래통제 오류	E-TCF-CTL-001	미등록 거래

Timeout 오류	E-TCF-TIME-0001	온라인 Timeout
SV 업무 오류	E-SV-BIZ-0001	Single View 업무 오류
OM 관리 오류	E-OM-BIZ-0001	운영관리 오류
JWT 인증 오류	E-JWT-AUTH-0001	토큰 검증 실패
파일 오류	E-UD-FILE-0001	파일 업로드 실패

11. 전체 네이밍 연결 예시

SV 고객요약 조회

항목	표준값
업무코드	SV
Context	/sv
WAR	sv.war

Module	sv-service
Package Root	com.nh.n sight.marketing. sv
ServiceId	SV.Customer.selectSummar y
거래코드	SV-INQ-0001
Handler	SvCustomerSummaryInquiry Handler
Facade	SvCustomerFacade
Service	SvCustomerService
Rule	SvCustomerEligibilityRul e
DAO	SvCustomerDao
Mapper Interface	SvCustomerRdwMapper
Mapper XML	mapper/sv/SvCustomerRdwM apper.xml
SQL ID	selectCustomerSummary

Result DTO `CustomerSummaryResult`

오류코드 `E-SV-BIZ-0001`

OM 사용자 조회

항목 표준값

업무코드 `OM`

Context `/om`

WAR `om.war`

Module `tcf-om`

Package Root `com.nh.nsight.om` 또는
`com.nh.nsight.marketing.om`

ServiceId `OM.User.inquiry`

거래코드 `OM-INQ-0001`

Handler `OmUserInquiryHandler`

Facade `OmUserFacade`

Service	<code>OmUserService</code>
DAO	<code>OmUserDao</code>
Mapper Interface	<code>OmUserMapper</code>
Mapper XML	<code>mapper/om/OmUserMapper.xml</code>
SQL ID	<code>selectUserList</code>
오류코드	<code>E-OM-BIZ-0001</code>

12. 금지 규칙

금지 항목	나쁜 예	표준
의미 없는 약어	<code>SvCustSvc</code>	<code>SvCustomerService</code>
계층명 누락	<code>SvCustomer</code>	<code>SvCustomerService,</code> <code>SvCustomerMapper</code>
DB와 업무 혼합 Mapper	<code>SvMapper</code>	<code>SvCustomerRdwMapper</code>
ServiceId와 Handler 불일치	<code>SV.Customer.selectSumma</code> <code>ry / SvSearchHandler</code>	<code>SvCustomerSummaryInquir</code> <code>yHandler</code>

SQL ID 중복	여러 Mapper에 <code>selectList</code> 만 사용	<code>selectCustomerList</code> , <code>selectCampaignList</code>
REST URL과 TCF URL 혼용	<code>/sv/customer/list</code> , <code>/sv/online</code> 혼재	기본은 <code>/sv/online</code>
Java SQL 직접 작성	<code>String sql = "SELECT..."</code>	Mapper XML
업무로직을 SQL에 숨김	CASE WHEN 남용	Rule/Service로 분리
운영 DB와 분석 DB Mapper 혼합	RDW/ADW SQL 한 Mapper에 혼재	<code>RdwMapper</code> , <code>AdwMapper</code> 분리

13. 최종 적용 기준

업무코드



Context / WAR / Module



Package



ServiceId



Handler



Facade



Service



Rule



DAO



Mapper Interface



Mapper XML



SQL ID



거래로그 / 오류코드 / 감사로그

이 기준으로 가면 소스, 배포, **MyBatis**, 운영관리, 거래로그, 오류코드가 모두 같은 이름 체계를 갖게 됩니다. 특히 **NSIGHT**는 TCF 기반 구조이므로 **ServiceId**와 **MyBatis SQL ID**를 반드시 운영 추적 가능한 이름으로 관리하는 것이 가장 중요합니다.